OCSI session 6

22 au 23 Octobre 2018 – Programmation CL Bases

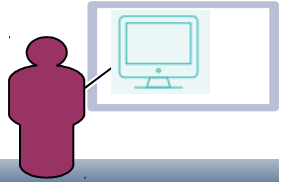
VOLUBIS
DEVENEZ UN EXPERT

Volubis.fr

Conseil et formation sur OS/400, I5/OS puis IBM ***i*** depuis 1994 !

Dans nos locaux, vos locaux ou par Internet

Marie Gris - mgris@volubis.fr



Programmation CL

Durée : 2 jours
Dates : du lundi 22 au mardi 23 octobre 2018
Intervenant : VOLUBIS
Sujets abordés :
- Principes (terminologie, concepts, principes de mise en œuvre)
- Déclaration et traitement d'un fichier écran
- Programmation structurée, expressions, fonctions intégrées
- Compilation, débogage
- Récupération d'attributs, DTAARA, conversion de dates, gestion des paramètres
- Gestion des messages
- Gestion des erreurs

Programmation CL

Principes

Les programmes qui s’exécutent sous IBM i peuvent être en CL(Langage de contrôle)

C’est un enchaînement de commandes de l’OS

Ces programmes permettent de :

- Mettre en place un environnement
- Modifier temporairement les caractéristiques d’un objet
- Automatiser des tâches
- Créer des scripts
- Enchaîner des programmes écrits dans différents langages

Programmation CL

Commande	libellé	cde inverse
ADDLIBLE	ajouter bibliothèque à *LIBL	RMVLIBLE
EDTLIBL	réviser *LIBL	
CRTLIB	créer une bibliothèque	DLTLIB
SAVLIB	sauvegarder	RSTLIB
WRKOBJ	obtenir une liste d'objet à gérer	
WRKLIB	" " " de bibliothèques	
WRKF	" " " de fichiers	
WRKLNK	gérer les liens des fichiers IFS (stream)	
CALL	lancer un programme	
GO	lancer un menu (GO ASSIST par ex.)	
SNDMSG	envoyer un message	
WRKMSG	gérer ses messages	
CHGMSGQ	modifier la réception des messages	
	(DLVRY(*BREAK ou *HOLD)	
Travaux		
WRKUSRJOB	gérer les travaux d'un utilisateur	
WRKJOBQ	gérer les travaux en attente	
WRKSBS	gérer les sous systèmes actifs	
WRKACTJOB	gérer les travaux actifs	
Spools		
WRKSPLF	gérer les états d'un utilisateur	
WRKOUTQ	gérer les files d'attentes imprimantes	
Editeurs		
WRKWTR	gérer les éditeurs d'imprimantes	

Programmation CL

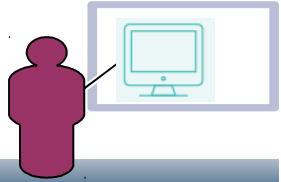
Il existe 2 façons d’écrire des scripts

En flot de commandes

- Non utilisé aujourd’hui
- Peut être stocké sur un support externe
- Les instructions utilisées dans ce flot ne sont pas interprétées
- Pas de compilation
- Si une instruction est en erreur passe à la suivante
- Pas de gestion des erreurs
- Pas de variables
- Type de source CL

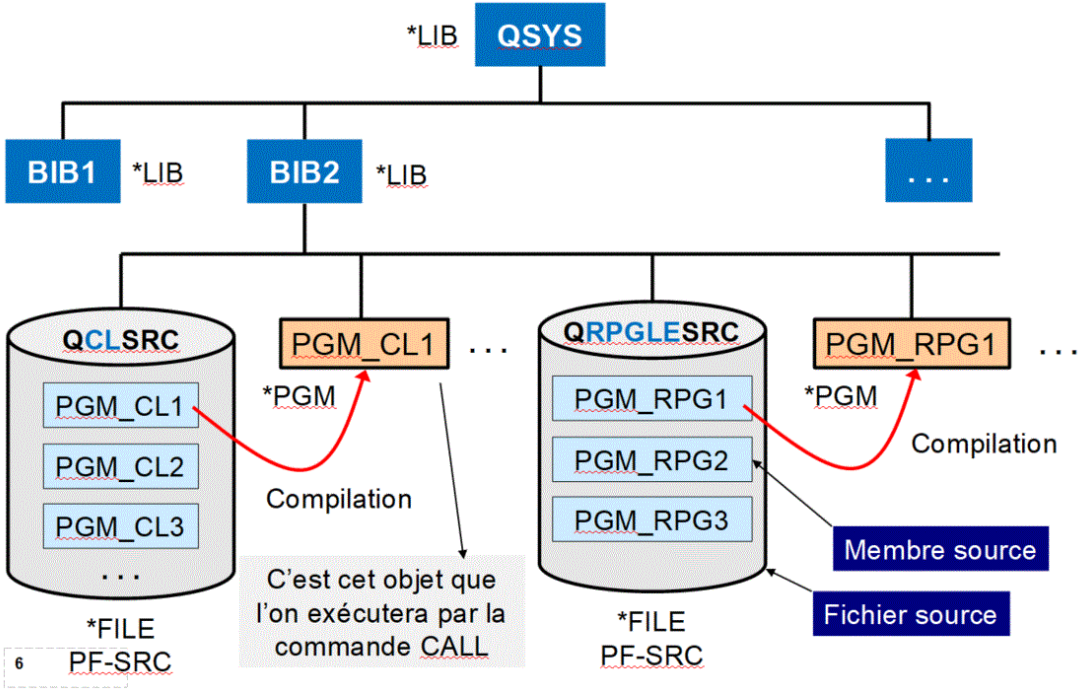
Dans un programme compilé

- C’est le résultat de la compilation qui sera exécuté
- Il faut générer un membre source, souvent sauvegardé dans un fichier source QCLSRC par défaut
- Ce membre sera de type CLP



Programmation CL

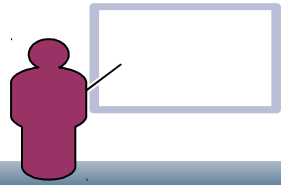
Fichiers source et membres source



Programmation CL

- Pour saisir un source lancez PDM

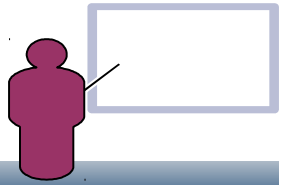
```
                         Gestionnaire de développement (PDM)

Choisissez l'une des options suivantes :

     1. Gestion des bibliothèques
     2. Gestion des objets
     3. Gestion des membres

     9. Gestion des options définies par l'utilisateur









Option ou commande
===> _
F3=Exit        F4=Invite        F9=Rappel          F10=Entrée de commandes
F12=Annuler    F18=Modifier valeurs par défaut
                                        (C) COPYRIGHT IBM CORP. 1981, 2007.
MA  +  A                                                          20/007
```



Programmation CL

- Sélectionnez le membre à explorer

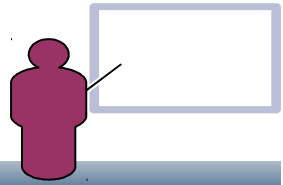
```
                              Sélection des membres

  Indiquez vos choix, puis appuyez sur ENTREE.

    Fichier . . . . . . . . .   QCLSRC      Nom, F4 pour liste

      Bibliothèque  . . . . .     FORMATION  *LIBL, *CURLIB, nom

    Membre :
      Nom . . . . . . . . . .   *ALL        *ALL, nom, *générique*
      Type  . . . . . . . . .   *ALL        *ALL, type, *générique*, *BLANK


  F3=Exit     F4=Invite     F5=Réafficher     F12=Annuler

MA  +  A                                                          05/032
```



Programmation CL

- Liste des membres du fichier source QCLSRC

```
                              PDM - Gestion des membres                AS400

Fichier . . . . .   QCLSRC
  Bibliothèque  .     FORMATION           Afficher à partir de .

Indiquez vos options, puis appuyez sur ENTREE.
 2=Editer   3=Copier   4=Supprimer   5=Afficher  6=Imprimer  7=Rebaptiser
 8=Description  9=Sauvegarder  13=Modifier texte 14=Compiler 15=Créer module...

Opt  Membre      Type        Texte
__   ATTENTE     CLP         Attente 3 HEURES
__   BOUCLE      CLP         pgm de boucle (envoi de messages)
__   BOUCLE2     CLP         pgm de boucle (calcul en boucle)
__   BOUCLE3     CLP         pgm de boucle (boucle 999999)
__   INITPGM     CLP         Ouverture de session
__   JOB1        CLP         soumission de SLEEP (ca "bouche" QGPMR) puis JOB1
__   JOB2        CLP         soumission de DLYJOB 3 après ENDSBS QPGMR
__   JOB3        CLP         soumission de PLANTE (pgm qui plante ==> MSGW)
                                                                  A suivre...
Paramètres ou commande
===>
F3=Exit     F4=Invite                F5=Réafficher        F6=Créer
F9=Rappel   F10=Entrée de commandes  F23=Autres options   F24=Autres touches
```

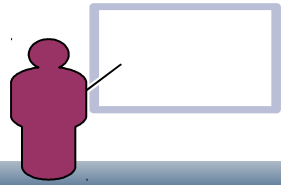


- Insertion, suppression, déplacement et copie de lignes
  - Par des codes (I, M, D…) qui se saisissent dans la colonne "Numéro de séquence" (numéro de ligne)

```
Colonnes  . . :    1  71  Edition
SEU==> ____________________________________
FMT ** ...+... 1 ...+... 2 ...+... 3 ...+
       ***************  Début des données
0001.00                PGM
0002.00                ENDPGM
       ***************  Fin des données
```

```
Colonnes  . . :    1  71  Edition
SEU==> ____________________________________
FMT ** ...+... 1 ...+... 2 ...+... 3 ...
       ***************  Début des données
0001.00                PGM
''''''''
0002.00                ENDPGM
       ***************  Fin des données
```

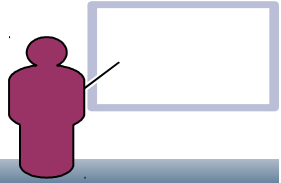
- **I :** pour insérer une ligne
- **C :** pour copier une ligne, sa destination est indiquée par A ou B (After ou Before) – CC / CC pour copier un bloc de lignes
- **M :** pour déplacer une ligne, sa destination est indiquée par A ou B MM / MM pour déplacer un bloc de lignes
- **D :** pour supprimer une ligne – DD / DD pour supprimer un bloc de lignes
- **RP :** pour répéter une ligne – RPn pour répéter n fois



Programmation CL

- Création d'un programme CL
  - Saisir les commandes CL de votre programme
    - PGM (Début de programme)
    - Les commandes CL à exécuter
      - Tapez la commande puis F4 pour afficher l'invite
    - ENDPGM (fin de programme)

```
 Colonnes  . . :    1  80                          Edition
 SEU==> ____________________________________________________
 FMT ** ...+... 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
        ***************  Début des données  ***************************************************
0001.00 PGM                                                                          180913
0002.00             SNDMSG     MSG('le programme fonctionne') +                      180913
0003.00                          TOUSR(MG)                                           181002
0004.00                                                                              180919
0006.00                                                                              180919
0007.00 ENDPGM                                                                       180913
        ****************  Fin des données  ****************************************************
```

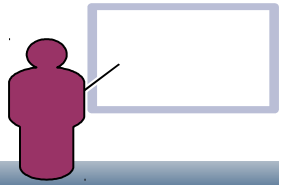


Programmation CL

- Création d'un programme CL
  - Saisir les commandes CL de votre programme
    - Si la commande apparaît en inversion d'image c'est que la syntaxe est erronée

```
 Colonnes  . . :    1  80                                Edition
 SEU==> ____________________________________________________________
 FMT ** ...+... 1 ...+... 2 ...+... 3 ...+... 4 .. .+... 5 ...+... 6 ...+... 7
        ************** Début des données ************************************
0001.00 PGM
0002.00                 SNDMGG     MSG('le programme fonctionne') +
0003.00                              TOUSR(MG)
0004.00
0006.00
0007.00 ENDPGM
```

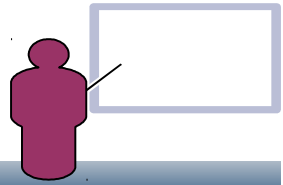
```
Commande SNDMGG non trouvée dans la bibliothèque *LIBL.
MA  +   A
```



Programmation CL

- Création d'un programme CL
  - Sortir de l'éditeur par F3
    - Sauvegardez votre source

```
                                   Sortie

 Indiquez vos choix, puis appuyez sur ENTREE.

   Modification/création du membre . .   O            O=Oui, N=Non
     Membre  . . . . . . . . . . . . .   TP01         Nom, F4 pour liste
     Fichier . . . . . . . . . . . . .   QCLSRC       Nom, F4 pour liste
       Bibliothèque  . . . . . . . . .     MG         Nom
     Texte  . . . . . . . . . . . . . .  le premier TP CL
__________
     Renumérotation  . . . . . . . . .   O            O=Oui, N=Non
       Début . . . . . . . . . . . . .   0001.00      0000.01-9999.99
       Incrément . . . . . . . . . . .   01.00        00.01-99.99

   Impression du membre  . . . . . . .   N            O=Oui, N=Non

   Retour à l'écran d'édition  . . . .   N            O=Oui, N=Non

   Affichage de la liste des membres . . N            O=Oui, N=Non
```



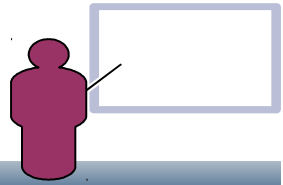
Programmation CL

- Création d'un programme CL
  - Compilez votre programme
    - Option 14 sur la ligne
    - Vérifiez vos informations de création de PGM si nécessaire (touche F4)

```
14   TP01        CLP        le premier TP CL
     TP01CLP     CLP        CL pour appel RPG TP01
```

```
                    Créer un programme CL (CRTCLPGM)

 Indiquez vos choix, puis appuyez sur ENTREE.

 Programme  . . . . . . . . . . . > TP01        Nom
   Biblio . . . . . . . . . . . . >   MG         Nom, *CURLIB
 Fichier source . . . . . . . . . > QCLSRC       Nom
   Biblio . . . . . . . . . . . . >   MG         Nom, *LIBL, *CURLIB
 Membre source  . . . . . . . . . > TP01         Nom, *PGM
 Texte 'descriptif' . . . . . . .   *SRCMBRTXT

                         Autres paramètres

 Remplacer programme  . . . . . . > *NO          *YES, *NO
```

```
La compilation du membre TP01 a été soumise par lots.
```



Programmation CL

- Création d'un programme CL
  - Compilez votre programme
    - La compilation est soumise en batch et génère un message

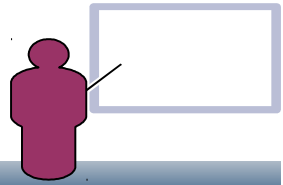
```
La compilation du membre TP01 a été soumise par lots.
```

    - Travail terminé signifie que le programme a été généré

```
Programme TP01 créé dans la bibliothèque MG.
```

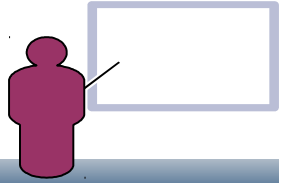
    - Vous retrouvez le résultat de la compilation dans votre outq (WRKSPLF)

```
 Fichier . . . . :   TP01
 Contrôle  . . . .   _______
 Recherche . . . .   ______________________________
 *...+....1....+....2....+....3....+....4....+....5....+....6....+....7....+....8....+.
  No SEQ  *...+... 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+..
     100- PGM
     200-              SNDMSG     MSG('le programme fonctionne') +
     300                            TOUSR(MG)
     400-
     500-
     600- ENDPGM
 * * * * *   F I N   D U   S O U R C E   * * * * *
  5770SS1 V7R3M0  160422           Langage de contrôle              MG/TP01
                                    Références croisées
 * CPD0792 10  Aucun label, variable ou zone de données utilisés dans ce programme.
 * * * * *   F I N   D E S   R E F E R E N C E S   C R O I S E E S    * * * * *
  5770SS1 V7R3M0  160422           Langage de contrôle              MG/TP01
                                    Récapitulatif des messages
                 Gravité
 Total           0-9  10-19  20-29  30-39  40-49  50-59  60-69  70-79  80-89  90-99
     1             0      1      0      0      0      0      0      0      0      0
 Programme TP01 créé dans bibliothèque MG le 02/10/18 à 11:06:24.
 Programme TP01 créé dans bibliothèque MG. Gravité d'erreur maximale : 10.

 F3=Exit   F12=Annuler   F19=Gauche   F20=Droite   F24=Autres touches
```



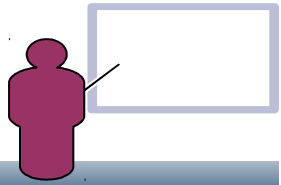
Programmation CL

- Un programme en langage de contrôle comporte 2 parties : Une partie déclarative, une partie traitement
- Structure d'un programme CL
  - **PGM** pour débuter le programme
  - DCLF pour déclarer les fichiers
  - **DCL** pour déclarer les variables
  - Des **commandes** CL, des traitements
  - **ENDPGM** pour terminer le programme
- Les fichiers et variables doivent être déclarés en début de programme
- Les lignes à blanc sont acceptées
- Les commentaires sont délimités par /* et */



Programmation CL

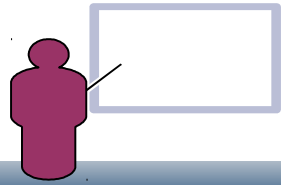
- Un programme en langage de contrôle comporte 2 parties : Une partie déclarative, une partie traitement
- Déclarations
  - La partie déclarative sert à déclarer les variables utilisées par votre programme et les fichiers utilisés lors du traitement spécifique sur un écran ou un fichier base de données
  - Déclaration de variable
    - Un programme qui utilise des variables doit indiquer leur longueur et leur type
    - Pour déclarer une variable il faut utiliser la commande **DCL**
      - Les valeurs alphanumériques se saisissent entre côtes ex (**‘**VALALPHA**’**)
      - Les valeurs numériques se saisissent directement ex (1234)
      - Le nom des variables commence par le caractère **&**



Programmation CL

Exemple: Pensez à faire F4

```
 DCL       VAR(&CTR) TYPE(*DEC) LEN(4 0) VALUE(1000)
                     !         !                    !                    !
           !         !                    !         !
----------------------         ! -------------------------             !
!Dans ce paramètre     ! Celui-ci définira sa                          !
!nous indiquerons le   ! longueur. Une seule                           !
!nom de la variable,   ! valeur pour les variables                     !
!qui doit toujours     ! alphanum. (9999 maxi) et                      !
!commencer par le      ! 2 pour le numérique                           !
!caractère &           ! (15 9 maxi)                                   !
                       !                                               !
   -------------------------------------                   ---------------------------
   !Dans celui-ci nous indiquerons        !  !  ce paramètre sert à    !
   !le type. 6 valeurs sont possibles:    !  !  définir une valeur      !
   !  *DEC  numérique packé               !  !  initiale à la variable.  !
   !  *CHAR alphanumérique                !  !  Il est, bien entendu,    !
   !  *LGL  logique ou booléen('0'/'1')   !  !  facultatif               !
   !  *INT  binaire signé (V5R30)         !
   !  *UINT binaire non signé (V5R30)     !
   !  *PTR  pointeur (très rare)          !
```



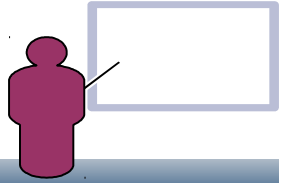
Programmation CL

- Déclaration de fichier

  Utile quand vous voulez lire un fichier, il n'est pas nécessaire de déclarer les fichiers que vous utilisez dans des commandes de copie, de remise à blanc, etc...

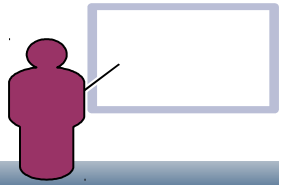
  – Pour déclarer un fichier il faut indiquer la commande **DCLF**

```
DCLF   FILE(*LIBL/FIC1) RCDFMT(FOR1)       OPNID(  )
            !                  !               !
   ----------------------      !               !
   Dans ce paramètre vous      !               Pour ouvertures multiples
   indiquerez le nom du        !                 en V5R30.
   fichier à utiliser,         !
   ainsi que sa bibliothèque   !
                  ----------------------------------------
                  Dans celui-ci vous indiquerez
                  le ou les formats utilisés (max. 50).
                  Si vous voulez les utiliser tous,
                  vous indiquerez *ALL (valeur par défaut)
```



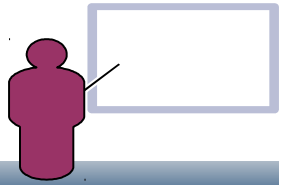
Programmation CL

- Déclaration de fichier
  - ATTENTION: Quand vous déclarez un fichier, le compilateur va déclarer automatiquement les zones de votre format.
  - Vous pourrez utiliser ces zones en indiquant leur nom externe, précédé du caractère **&** .
  - Toutes les variables du langage de contrôle commencent par **&**.



Programmation CL

- Traitement
  - Partie où vous indiquerez les instructions à exécuter
  - Vous pourrez effectuer des calculs, des comparaisons, des déplacements de zone entière ou partie
  - CHGVAR
    - Permet de modifier une variable
      - CHGVAR VAR(&A) VALUE(0) /*Mettre 0 dans la variable A*/
      - CHGVAR VAR(&A) VALUE(&B) /* Mettre la variable B dans A*/

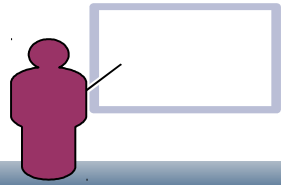


Programmation CL

- Traitement
  - Calculs
    - La plupart des opérations arithmétiques sont possibles en langage de contrôle
    - Pour effectuer l’addition A = A+2
      - Vous allez utiliser la commande qui permet d’affecter une nouvelle valeur à une variable

```
    CHGVAR     VAR(&A) VALUE(&A + 2)
         !           !             !
         !           !             !
------------------   !  ------------------------
Instruction qui      !  Nouveau contenu de la
permet de changer    !  variable. Ici le résultat
le contenu d'une     !  d'une addition.
variable.            !
            ------------------
            Variable dont le
            contenu doit être
            changé
```

Cette commande CHGVAR peut être utilisée pour affecter une constante à une variable, on l'utilisera à chaque fois que l'on voudra changer le contenu d'une variable.



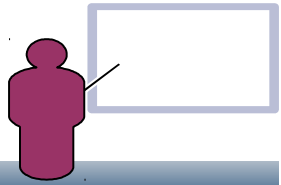
Programmation CL

- Traitement
  - Calculs

On peut effectuer des calculs complexes grâce à cette commande.

Sachez que pour indiquer une addition on indiquera : +
une soustraction : -
une division : /
une multiplication : *

```
                B + C - 3
 exemple : A = ----------
                    X
               ----------
               (Z - 3)* 5

S'écrira : CHGVAR    VAR(&A) VALUE((&B +&C -3) / (&X / ((&Z - +
                       3)*5)))                                 ^
                                                               |
                                                   ------------------
                                                   + signifie que la
                                                    commande continue
                                                    ligne suivante .
```



Programmation CL

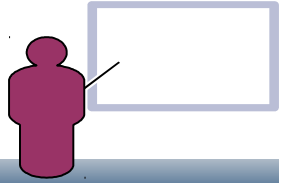
- Traitement
  - Concaténation

Il est de possible de regrouper plusieurs zones entre elles.
Pour cela nous utilisons l'instruction ***CAT** . Si nous voulons regrouper des zones entre elles, en insérant 1 blanc entre chaque zone, nous utilisons ***BCAT**. Si nous voulons supprimer les blancs de droite de la première zone nous utilisons ***TCAT**.
Dans le paramètre VALUE saisir l'expression entre parenthèses

```
CHGVAR     VAR(&DEPART) VALUE(&HH *CAT &MM *CAT &SS)
```

Exemple:

```
 PGM
        DCL        VAR(&A) TYPE(*CHAR) LEN(6) VALUE('BON   ')
        DCL        VAR(&B) TYPE(*CHAR) LEN(6) VALUE('JOUR  ')
        DCL        VAR(&C) TYPE(*CHAR) LEN(12)
-------CHGVAR     VAR(&C) VALUE(&a *CAT &b)
!      CHGVAR     VAR(&C) VALUE(&a *TCAT &b)----------------
!                                                           !
------> la variable &C aura le contenu 'BON   JOUR  '       !
                                                            !
        la variable &C aura le contenu 'BONJOUR     ' <-----
```



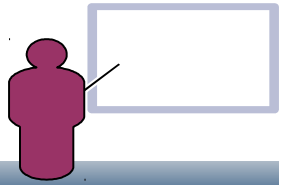
Programmation CL

- Traitement
  - Extraction

Nous pouvons aussi extraire d'une chaîne de caractères, certains des caractères qui la composent, ceci grâce à l'instruction %SST (ou %SUBSTRING)

Exemple:

```
          PGM
          DCL        VAR(&A) TYPE(*CHAR) LEN(15) VALUE('BON APPETIT')
          DCL        VAR(&B) TYPE(*CHAR) LEN(7)
        /*                                                        */
        /* JE VEUX PLACER LA VALEUR APPETIT DANS LA VARIABLE &B */
        /*                                                        */
          CHGVAR     VAR(&B) VALUE(%SST(&A 5 7))
            !                        !  !-----!
----------------                     !      !
modif. d'une                         !      !
variable                 ----------------   !
                         extraction d'une   ---------------------------
                         sous-chaîne de     Variable extraite depuis la
                         caratères          position 5 sur 7 caractères
```



Programmation CL

- Traitement
  - Extraction

Il est possible de ne modifier qu'une partie d'une variable :

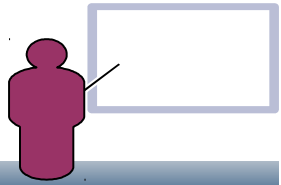
```
PGM
     DCL        VAR(&A) TYPE(*CHAR) LEN(15) VALUE('BON APPETIT')
     DCL        VAR(&B) TYPE(*CHAR) LEN(15) VALUE('JOYEUX NOEL')

     CHGVAR     VAR(%SST(&B 8 7)) VALUE(%SST(&A 5 7))
```

La partie VALUE du CHGVAR peut comprendre des sous-chaînes concaténées.

```
CHGVAR     VAR(&A) VALUE(%sst(&B 1 2) *CAT %sst(&C 3 2))
```

Exemple :

```
PGM
     DCL        VAR(&STAG) TYPE(*CHAR) LEN(10) VALUE('FORMATION1')
     DCL        VAR(&BD)   TYPE(*CHAR) LEN(10) VALUE('BDVIN')

     CHGVAR VAR(%sst(&BD 6 1) VALUE(%sst(&STAG 10 1)
     ADDLIBLE &BD /* Ajout de BDVIN1  */
```



Programmation CL

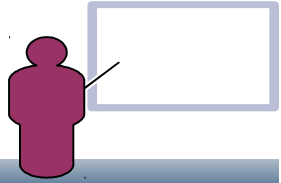
- Traitement
  - Opérations logiques

En langage de contrôle il est possible d'effectuer des comparaisons entre deux variables, ou entre une variable et une constante, et suivant le résultat de cette comparaison déclencher une action.

Supposons le test suivant si A > B appeler le prog. PROGA,
sinon appeler le prog. PROGB

Cela s'écrira en langage de contrôle :

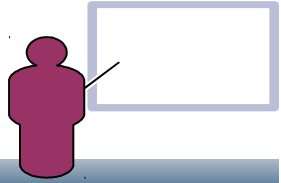
```
IF          COND(&A *GT &B) THEN(CALL PGM(PGMA))

ELSE        CMD(CALL PGM(PGMB))
```



Programmation CL

- Traitement
  - Opérations logiques
    - Reprenons l'instruction IF et détaillons-la :

```
          IF         COND(&A *GT &B) THEN(CALL PGM(PGMA))
          !                   !            !
--------------                !    ---------------------------
instruction de                !     on place ici l'instruction
comparaison                   !     à exécuter si la condition
                              !     est remplie
                     ------------------
                      Opérateur
                      relationnel
                      *GT ( >  )
                      *GE ( >= )
                      *LT ( <  )
                      *LE ( <= )
                      *EQ ( =  )
                      *NE ( ^= )
```



Programmation CL

- Traitement
  - Opérations logiques
    - on peut donner des conditions complexes avec `*AND ( & )`

      `*OR ( ! )`

      `*NOT ( ^ )`

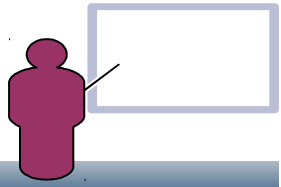
      sous la forme :

      ```
      IF COND((&A *EQ 0) *OR (&x *LT &Y)) THEN( ...)
      ```

Attention sur l'invite il faut saisir toutes les parenthèses :

si (IF)

condition . . . . . **(**((&a *eq 0) *or (&x *lt &y)**)**



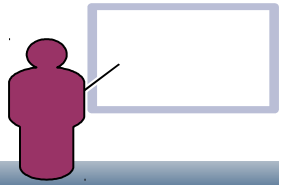
Programmation CL

- Traitement
  - Opérations logiques
    - Continuons avec ELSE:

```
ELSE        CMD(CALL PGM(PGMB))
                      !
                 -------------------------
                 instruction à réaliser
                 si la condition n'est pas
                 remplie
```

- Supposons, si la condition est remplie, que je veuille exécuter plusieurs commandes, je ne peux placer qu'une seule instruction derrière THEN.
- Pour cela j'utiliserai les commandes DO et ENDDO.
- EX: si A > B alors appel PGMA1 PGMA2 PGMA3

  sinon appel PGMB1 PGMB2 PGMB3



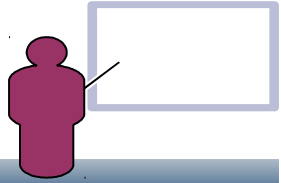
Programmation CL

- Traitement
  - Opérations logiques

```
IF         COND(&A *GT &B) THEN(DO)--> début des instructions
 CALL PGMA1
 CALL PGMA2
 CALL PGMA3
ENDDO -->   marque la fin des instructions à exécuter
ELSE       CMD(DO)
 CALL PGMB1
 CALL PGMB2
 CALL PGMB3
ENDDO
```

On peut aussi utiliser des IF imbriqués (peu lisible)

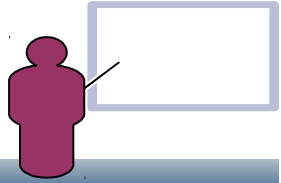
```
IF  ------- THEN(IF ------ THEN(IF_____THEN(___) ... )
```



Programmation CL

- Traitement
  - Opérations logiques
    - Priorité des opérateurs

Priorité	Opérateur
Haute	Parenthèses
	Signes *NOT
	Multiplication Division
	Addition Soustraction
	Concaténation
	Comparaison
	*AND
	*OR
	De gauche à droite



Programmation CL

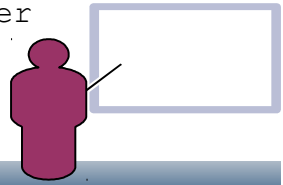
- Les branchements inconditionnels
  - A tout moment dans un programme CL il est possible de se débrancher à une autre séquence d'instruction.

  - Supposons le programme d'enchaînement des tâches suivant :

    - exécution de PGMA
    - tant que A = B exécution de PGMB et PGMC
    - exécution de PGMD

    S'écrira :

```
              CALL       PGM(PGMA)
 ENCORE:      CALL       PGM(PGMB)
   ^          CALL       PGM(PGMC)
   !          IF         COND(&A = &B) THEN(GOTO ENCORE)
   !          CALL       PGM(PGMD)          !
   !                      ------------------------------------
--------------------       GOTO instruction de débranchement
définition d'une           inconditionnel. A comme paramètre une
étiquette                  étiquette qui indique l'endroit où se débrancher
```



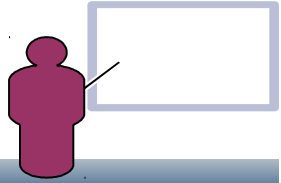
Programmation CL

- Les boucles
  - La version 5.30 de l'OS/400 apporte de nombreuses améliorations au CL.
    particulièrement les boucles (Do-While et Do-Until) et l'ordre Select.
    Nous avons donc, maintenant :

    - **DOWHILE** COND(même test que sur un IF)
      …
    - ENDDO
      - c'est un vrai dowhile, le test est réalisé avant (on peut ne jamais entrer dans la boucle) et il s'agit de la condition pour FAIRE

    - **DOUNTIL** COND(même test que sur un IF)
      …
    - ENDDO
      - pour dountil, le test est réalisé sur le ENDDO (on fait au moins une fois) et la condition donnée est celle pour SORTIR.

    - **DOFOR** VAR(&cpt) FROM(1) TO(22) BY(3)
      …
    - ENDDO



Programmation CL

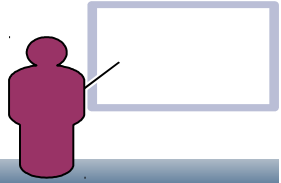
- Les boucles
  - pour ces trois boucles on peut forcer

    - une sortie anticipée par **LEAVE**
      sans paramètre, on sort de la boucle en cours (la dernière)
      on peut mettre un LABEL devant le DOxxx et indiquer le label sur le LEAVE

    - un saut d'un tour de boucle par **ITERATE**

  - et enfin, l'équivalent du CASE SQL ou du SELECT RPG :

```
SELECT
 WHEN COND( ) THEN( )   /* seul le premier test vrai est traité */
 WHEN COND( ) THEN()
 OTHERWISE CMD( )   /* Facultatif */
ENDSELECT
```

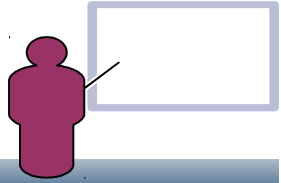


Programmation CL

- Les sous programmes (depuis la V5R4)
  - les sous programmes doivent être placés en **FIN** de pgm
    - un sous programme peut lancer un sous programme (99 niveaux possibles)
    - exemple de source CL avec sous programme

```
PGM
   DCL &V1 *CHAR LEN(20)
   DCL &rt *INT  LEN(4)

     .../...

     CALLSUBR SUBR(TEST) RTNVAL(&RT)


 /* fin logique du pgm */
 SUBR SUBR(TEST)

  .../...
 ENDSUBR RTNVAL(&rt)
```

```
##############################
#                            #
# La notion de valeur retour #
# est facultative            #
#                            #
# Si vous l'utilisez il doit #
#  s'agir d'une variable *INT #
#  sur 4 octets.             #
#                            #
##############################
```



Programmation CL

- Les fonctions intégrées

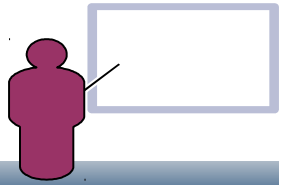
%SST	extraction d'une chaîne de caractères
%BIN	extraction et conversion d'une sous-chaîne en binaire
%SWITCH	manipulation des switchs du job (concept d'origine 36)
%OFFSET	manipulation d'un pointeur (incrémentation/déplacement)
%ADDRESS	assignation d'un pointeur avec l'adresse d'une variable

La PTF SI48166 (V7) propose

%TRIM	élimination des blancs d'extrémité d'une chaîne
%TRIMR	élimination des blancs de droite
%TRIML	élimination des blancs de gauche

La PTF SI49061 (V7) propose

%CHECK	vérification des caractères d'une variable(gauche->droite)
%CHECKR	vérification des caractères d'une variable(droite->gauche)
%SCAN	recherche d'une chaîne dans une variable



Programmation CL

- Les fonctions intégrées

Enfin, en version 7.2

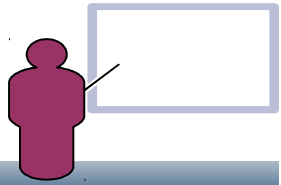
Fonctions de changement de genre	Gestion de la casse	Gestion des tailles des variables
%CHAR	%LOWER	%LEN
%DEC	%UPPER	%SIZE
%INT		
%UINT		

Exemples:

```
CHGVAR VAR(&MSG) VALUE(%lower(&MSG))
CHGVAR VAR(&LG)  VALUE(%len(&MSG))
```

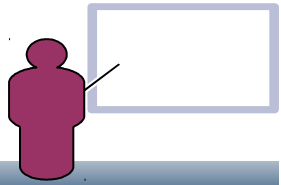
Puis, en 7.3 (SI67797)

```
%PARMS , retourne le nombre de paramètres reçus
```



Programmation CL

- Fichier écran
  - Le langage CLP permet d'utiliser un fichier écran avec des opérations d'écriture ou de lecture, et les fichiers bases de données seulement en lecture.
  - Déclaration d'un fichier écran
    - `DCLF FILE(nom du fichier) RCDFMT(nom du ou des formats)`
    - Cette commande permettra aussi de déclarer les zones du fichier automatiquement, avec des noms de zone commençant par &.
    - Dans le cas de fichier écran il déclarera les indicateurs en tant que variables logiques et leur donnera un nom &INXX
      - XX étant le numéro de l'indicateur utilisé.



Programmation CL

- Fichier écran

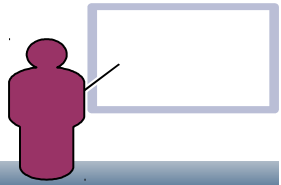
  Les opérations d'E/S en langage de contrôle _____

  - Lecture

    RCVF (RECEIVE FILE)

```
RCVF        DEV(*FILE) RCDFMT(nom du format) WAIT(*YES)
```

N.B. Laissez les valeurs par défaut pour un fichier base de données.



Programmation CL

- Fichier écran

  Les opérations d'E/S en langage de contrôle

- Écriture (uniquement pour les fichiers écran)

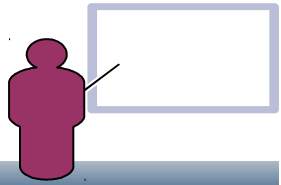
  SNDF (SEND FILE)

  SNDF RCDFMT(Nom du format)

- Écriture puis lecture(uniquement pour les fichiers écran)

  SNDRCVF (SEND RECEIVE FILE)

  SNDRCVF RCDFMT(Nom du format)

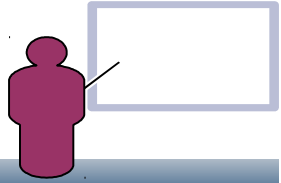


Programmation CL

- Fichier écran

  Utilisation d'un DSPF

```
!                                                                !
!                    LANCEMENT DES TRAVAUX DU SOIR               !
!                                                                !
!                                                                !
!                                                                !
!                                                                !
!                                                                !
!          ENTRER L'HEURE DE DEBUT DE LA CHAINE  __ __ __        !
!                                                (HH MM SS)      !
!                                                                !
!                                                                !
!                                                                !
!                                                                !
!                                                                !
!   F3  = Sortie.                                                !
```



Programmation CL

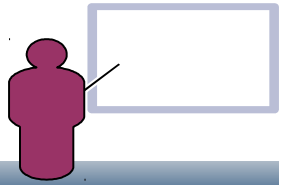
- Fichier écran

Codification du fichier écran

Codification du fichier écran

```
A*            10:03:22     FORMATION    REL-R02M00  5728-PW1
A                                        PRINT
A          R HEURF1
A*
 *  ca03 = F3 autorisé
A                                        CA03(03 'fin de travail')
 * divers constantes
A                                    3 27'LANCEMENT DES TRAVAUX DU SOIR'
A                                   10 14'ENTRER L'''
A                                   10 23'HEURE DE DEBUT DE LA CHAINE'
 * les trois variables (deviendront &HH, &MM et &SS en CL)
A*
A            HH             2   I 10 54DSPATR(MDT)
A                                        CHECK(RB)
A                                        RANGE('18' '24')
A                                        DFT('00')
A            MM             2   I 10 57DSPATR(MDT)
A                                        CHECK(RB)
A                                        RANGE('00' '60')
A                                        DFT('00')
A            SS             2   I 10 60DSPATR(MDT)
A                                        CHECK(RB)
A                                        RANGE('00' '60')
A                                        DFT('00')
 * pied d'écran
A                                   11 53'(HH MM SS)'
A                                   19  6'F3 = sortie'
```

Le programme SOIRLANC

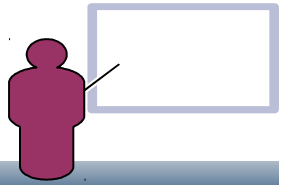


Programmation CL

- Fichier écran

Programme rattaché _____

```
            PGM
            DCL        VAR(&DEPART) TYPE(*CHAR) LEN(6)
            DCLF       FILE(HEURE)
            SNDRCVF    RCDFMT(HEURF1)
            IF         COND(&IN03) THEN(GOTO CMDLBL(FINCLP))
            CHGVAR     VAR(&DEPART) VALUE(&HH *CAT &MM *CAT &SS)
            SBMJOB     JOB(ATTENTE) JOBD(SOIR) CMD(DLYJOB +
                                                   RSMTIME(&DEPART) )
            SBMJOB     JOB(JOBSOIR) JOBD(SOIR) CMD(CALL PGM(SOIR))
FINCLP:     ENDPGM
```

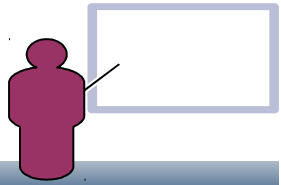


Programmation CL

- Fichier
  - Les fichiers en CL sont aussi souvent utilisés en lecture, suite à une commande plaçant un résultat dans un fichier de sortie.

```
Exemple : DSPOBJD OBJ(mabib/*ALL) OBJTYPE(*PGM) OUTPUT(*OUTFILE) +
            OUTFILE(QGPL/LISTEPGM)
```

  - Suite à cette commande, le fichier LISTEPGM peut être lu pour traitement.
    - DSPFFD pour connaitre la liste des zones
    - ou SELECT * from LISTEPGM sous SQL



Programmation CL

- Fichier

Avant la V5R30, nous ne pouvions déclarer qu'un seul fichier par pgm

ce qui fait qu'il n'était pas utile de préciser le nom .

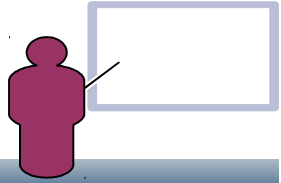
La v5r30 apporte un nouveau paramètre OPNID() sur les commandes DCLF

RCVF, SNDRCVF, SNDF permettant de manipuler plusieurs fichiers.

```
   DCLF       FILE(ECRAN1) OPNID(ECRAN)
.......................................................................
:  ATTENTION : les variables seront préfixées par l'OPNID suivi de "_" :
:                                                                      :
:   par exemple ce DSPF contient :                                     :
:                                                                      :
:   JOB -> nommée dans le pgm &ECRAN_JOB                               :
:   Indicateur 3 -> nommé &ECRAN_IN03                                  :
:......................................................................
```

l'OPNID est facultatif, mais un SEUL fichier peut ne pas en avoir, et nous sommes limités à 5 fichiers en tout, dans un même programme.

les commandes de manipulation de fichier (SNDF, RCVF et SNDRCVF) doivent utiliser le paramètre OPNID s'il a été utilisé lors de la déclaration.



Programmation CL

- Fichier

Avant la V5R30, nous ne pouvions déclarer qu'un seul fichier par pgm

ce qui fait qu'il n'était pas utile de préciser le nom .

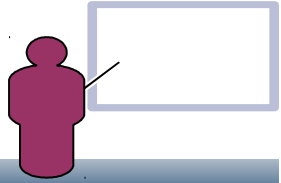
La v5r30 apporte un nouveau paramètre OPNID() sur les commandes DCLF

RCVF, SNDRCVF, SNDF permettant de manipuler plusieurs fichiers.

```
   DCLF       FILE(ECRAN1) OPNID(ECRAN)
.......................................................................
:  ATTENTION : les variables seront préfixées par l'OPNID suivi de "_" :
:                                                                      :
:   par exemple ce DSPF contient :                                     :
:                                                                      :
:   JOB -> nommée dans le pgm &ECRAN_JOB                               :
:   Indicateur 3 -> nommé &ECRAN_IN03                                  :
:......................................................................
```

l'OPNID est facultatif, mais un SEUL fichier peut ne pas en avoir, et nous sommes limités à 5 fichiers en tout, dans un même programme.

les commandes de manipulation de fichier (SNDF, RCVF et SNDRCVF) doivent utiliser le paramètre OPNID s'il a été utilisé lors de la déclaration.



Programmation CL

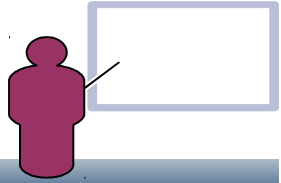
- Récupération d’attributs

  - Les commandes RTVxxx permettent de récupérer des informations / attributs du système, dans des variables du programme. Par exemple :
    - La commande **RTVUSRPRF** permet de récupérer des informations sur un profil utilisateur :
      - Quelle est sa CURLIB ?
      - Quel est son menu initial ? etc.

    - La commande **RTVJOBA** permet de récupérer les attributs du travail en cours :
      - Quel est l'utilisateur qui exécute le programme ? (l'utilisateur en cours)
      - Sur quel écran est-il connecté ?
      - Quelle est sa LIBL ? etc.

    - La commande **RTVOBJD** permet de récupérer les propriétés d'un objet :
      - Quel est le propriétaire de l'objet ?
      - A quelle date a-t 'il été créé ? modifié ? utilisé pour la dernière fois ? etc.

- Pour connaître toutes les commandes RTV : RTV* ou **GO CMDRTV**



Programmation CL

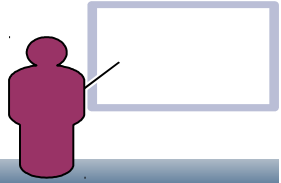
- Récupération des informations pour le travail en cours
  - A tout moment dans un programme en langage de contrôle il est possible de connaître l'environnement du travail. à quelle OUTQ le travail est-il rattaché ? Quel est le profil utilisateur en cours ? etc…
    - La commande qui nous permet cela est RTVJOBA.

Pensez à faire F4 puis F1 sur chacun des paramètres pour en retrouver le type, la longueur et l'usage

```
RTVJOBA     JOB(&NOMJOB) USER(&NOMUSER) +
                         OUTQ(&NOMOUTQ) ....
/* on ne renseigne que les paramètres dont on a besoin */
```

RTVJOBA va placer le(s) paramètre(s) recherché(s) dans la/les variable(s)

```
RTVJOBA   JOB(variable qui recevra le nom du TRAVAIL)
          USER(variable qui recevra le nom du PROFIL utilisateur)
          NBR(variable qui recevra le NUMERO du travail)
          OUTQ(variable qui recevra le nom de l'OUTQ)
          TYPE(variable qui recevra le type batch/interactif)
          DATE(variable qui recevra la DATE)
          USRLIBL(variable qui recevra *LIBL)
```

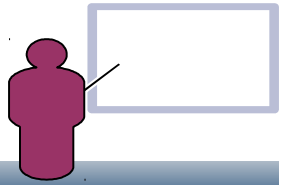


Programmation CL

- Récupération des informations pour le travail en cours

Exemple d’utilisation

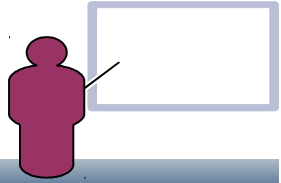
```
**************  Début des données  ********************************
PGM
             DCL        VAR(&USRLIBL) TYPE(*CHAR) LEN(2750)
             DCL        VAR(&POS) TYPE(*INT) LEN(4)
             DCL        VAR(&USER) *CHAR(10)
             DCL        VAR(&BIB) TYPE(*CHAR) LEN(10) VALUE(BDVIN)

             RTVJOBA    USER(&USER) USRLIBL(&USRLIBL)
             CHGVAR VAR(%sst(&BIB 6 1) VALUE(%sst(&USER 10 1)

             CHGVAR     VAR(&POS) VALUE(%SCAN(&BIB &USRLIBL))
             IF         COND(&POS = 0) THEN(DO)
               ADDLIBLE &BIB
             ENDDO
ENDPGM
***************  Fin des données  *********************************
```



Programmation CL

- Utilisation des DTAARA

  - Une Data area est un objet utilisé pour transmettre des données
    - Entre programmes d'un même job (*LDA)
    - Entre des travaux différents, asynchrones (Data area nommée, comme une variable permanente
    - Une data area ne peut être utilisée que pour stocker des informations à taille limitée et fixe



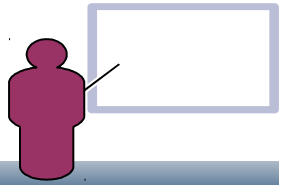
Programmation CL

- Utilisation des DTAARA
  - Il existe 2 types de data area
    - Local data area (LDA)
      - Elle est créée automatiquement par le système pour chaque job
        - Elle permet seulement la communication entre programmes d'un job
        - Toutefois un job soumis (SBMJOB) aura comme valeur initiale de sa LDA une copie de la LDA du job qui le soumet
      - La longueur de la LDA est 1024 caractères (*CHAR)
      - Exemple d'utilisation

```
PGM
        DCL         VAR(&DEPART) TYPE(*CHAR) LEN(6)
        DCLF        FILE(HEURE) RCDFMT(*ALL)

        SNDRCVF     RCDFMT(HEURF1)

        IF          COND(&IN03) THEN(GOTO CMDLBL(FINCLP))

        CHGVAR      VAR(&DEPART) VALUE(&HH *CAT &MM *CAT &SS)
        CHGDTAARA   DTAARA(*LDA (1 6)) VALUE(&DEPART)

        SBMJOB      JOB(TRAVSOIR) CMD(CALL SOIR)

   FINCLP:   ENDPGM
```

Elle sera utilisée dans le programme suivant (SOIR) par

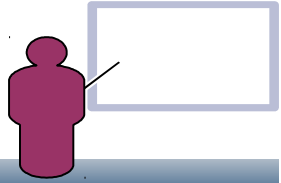
```
RTVDTAARA  DTAARA(*LDA (1 6)) RTNVAR(&DEPART)
```



Programmation CL

- Utilisation des DTAARA
  - Il existe 2 types de data area
    - La data area créée par le programmeur
      - Elle est créée avant d'être utilisée (CRTDTAARA)
        - Elle est commune à l'ensemble des jobs (stockée sur disque)
        - Elle permet donc la communication entre des jobs différents

```
CRTDTAARA DTAARA(Bibliothèque/nom de la DTAAREA)
          TYPE(*CHAR ou *DEC ou *LGL)
          LEN(2000 maxi si *CHAR
              24 dont 9 maxi si *DEC )
          VALUE(valeur initiale de la DTAARA)
          TEXT('texte d'information')
```



Programmation CL

- Utilisation des DTAARA
  - Il existe 2 types de data area
    - La data area créée par le programmeur

      Exemple d'utilisation à partir de l'exemple précédent

      Création d'une data area appelée SOIR

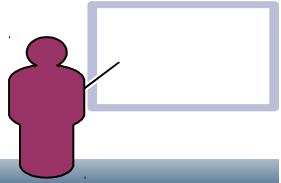
```
CRTDTAARA  DTAARA(EXEMPLE/SOIR) TYPE(*CHAR) LEN(6)
```

Programme de lancement

```
            PGM
            DCLF        FILE(HEURE) RCDFMT(*ALL)
            DCL         &DEPART *CHAR 6
            SNDRCVF     RCDFMT(HEURF1)
            IF          COND(&IN03) THEN(GOTO CMDLBL(FINCLP))
            CHGVAR      VAR(&DEPART) VALUE(&HH *CAT &MM *CAT &SS)
            CHGDTAARA   DTAARA(SOIR) VALUE(&DEPART)

            SBMJOB      JOB(TRAVSOIR) CMD(CALL SOIR)
 FINCLP:    ENDPGM
```

Programme appelé

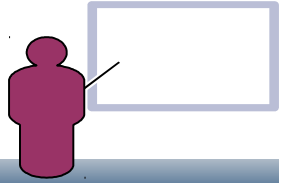
```
PROGRAMME SOIR
            PGM
            DCL         &SOIR *CHAR 6
            RTVDTAARA   DTAARA(SOIR) RTNVAR(&SOIR)
```



Programmation CL

- Conversion de dates
  - Plusieurs commandes permettent de récupérer des dates
    - RTVJOBA DATE()
    - RTVSYSVAL QDATE
    - RTVOBJD
  - Les dates récupérées sont dans des formats différents (utilisez l'aide sur la commande d'extractions pour plus d'informations)
  - Pour comparer 2 dates en CLP elles doivent être dans le même format
  - Vous pouvez utiliser la commande **CVTDAT** (convertir le format de date)

```
                    Convertir le format de date (CVTDAT)

Indiquez vos choix, puis appuyez sur ENTREE.

Date à convertir . . . . . . . . DATE          _
Variable pour date convertie . . TOVAR
Format date d'origine  . . . . . FROMFMT       *JOB
Format date de destination . . . TOFMT         *JOB
Séparateur date de destination   TOSEP         *JOB
```



Programmation CL

- Gestion des paramètres

  - Un programme CL peut être appelé par un programme écrit en n'importe lequel des langages, et peut donc recevoir des paramètres.

  - Pour recevoir les paramètres nous les indiquerons dans la commande PGM.

```
PGM        PARM(&A &B etc....)
```

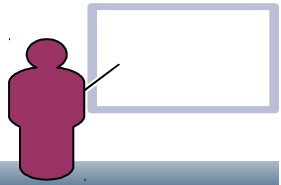
  - Il ne faudra pas oublier de définir les paramètres reçus.

  - Le programme retournera au programme appelant par l'instruction **ENDPGM** ou l'instruction **RETURN**, celui retrouvant ces paramètres, modifiés.

  - **ENDPGM** et **RETURN** entraînent la désactivation du programme appelé.

  - Un programme en CL peut bien sûr appeler un pgm et lui transmettre des données grâce à la commande CALL:

```
CALL       PGM(PGM1) PARM(&A &B etc...)
```



Programmation CL

- Passage des paramètres

  - Attention aux passages de constantes.

    - Sur AS/400 le passage de paramètres est un passage d'adresses.

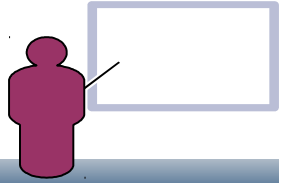
      - dans le cas de variables on passe l'adresse de la variable (pgm appelant)

      - dans le cas de constantes, les constantes sont stockées dans un espace de travail et l'on passe cette adresse

      - Mais quel espace réserver pour le stockage de ces constantes ???

    - Le CL a des valeurs par défaut, qui sont :

```
 APLHA si < à 32 ==> 32 octets
    sinon     ==> la longueur exacte de la constante.

NUMERIQUE       ==> 15 dont 5 !!!!
```



Programmation CL

- Gestion des messages
  - Vous pouvez bien sur envoyer un message à un utilisateur (par ex. QSYSOPR)
    ```
    SNDMSG MSG('votre texte') TOUSR(leprofil)
    ```
  - Ou sur un terminal en imposant une visualisation immédiate (BREAK)
    ```
    SNDBRKMSG MSG('votre texte') TOMSGQ(leterminal)
    ```
  - Mais vous pouvez aussi envoyer un message au pgm qui a demandé le lancement de votre CL. Si celui ci contient une ligne de commande, vous verrez votre message sur la dernière ligne de l'écran une fois le pgm terminé .
    ```
    SNDPGMMSG MSG('votre texte') MSGTYPE(*INFO)
    ```

- Enfin vous pouvez poser une question par **SNDUSRMSG** (PGM CL uniquement)

```
PGM
DCL &REP *CHAR 1
SNDUSRMSG MSG('Voulez vous voir vos spools ?') VALUES(O N) +
DFT(N) TOMSGQ(*) MSGRPY(&REP)
IF (&REP = 'O') THEN(DO)
WRKSPLF
ENDDO
ENDPGM
```

```
+-------------------------------------------------------------------------------
!                              Messages du programme
!
! Travail 028336/CM/PCACSA démarré le 27/09/18 à 10:01:40 dans le sous-système
! Voulez vous voir vos spools ?
!
!
! .../...
!
!
! Répondez, puis appuyez sur ENTREE.
!   Réponse . .
! ------------------------------------------------------------------------
!
!
```

Programmation CL

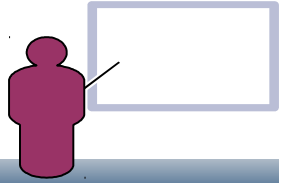
- Différents types de messages
  - *INFO : une simple information
  - *COMP : Achèvement => j'ai bien fait ce que je devais faire !
  - *DIAG : Diagnostique, en général suivi d'une erreur.
  - *ESCAPE : Erreur provoquant l'arrêt du pgm.

- Un message d'erreur doit faire référence à un message venant d'un fichier message (pour pouvoir être monitoré)
  - QCPFMSG contient un message CPF9898 pour vous aider, qui possède une seule variable:
    - le texte à afficher.

```
SNDPGMMSG MSGID(CPF9898) MSGF(QCPFMSG) MSGDTA('votre texte')
          MSGTYPE(*ESCAPE)
```

- *STATUS : Etat d'avancement, doit utiliser un ID message (CPF9897), apparaît <u>pendant le traitement</u> si envoyé à *EXT.

```
SNDPGMMSG MSGID(CPF9897) MSGF(QCPFMSG) MSGDTA('votre texte')
          TOPGMQ(*EXT) MSGTYPE(*STATUS)
```



Programmation CL

- Interception des messages d’erreur
  - Un programme en langage de contrôle peut se terminer anormalement à l'exécution d'une commande. Si c'est le cas, un message est envoyé par le système au programme, ce qui provoque son arrêt.
  - Si ce programme est une chaîne de traitements importants, il vaut mieux éviter l'arrêt pour des raisons bénignes.

  - Soit un programme qui ajoute une entrée dans la liste de bibliothèques

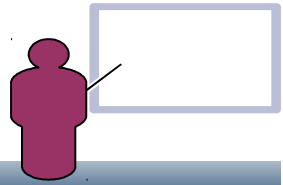
```
**************  Début des données  *****************
PGM
                ADDLIBLE   LIB(FORMATION)
ENDPGM
***************  Fin des données  ******************
```

  - Sans interception de message d’erreur, au 2ème appel nous aurons un message

```
                          Messages du programme

 Travail 035187/MG/PCMGA démarré le 04/10/18 à 10:20:39 dans le sous-système
 Message CPF2103 reçu par GESTERRCL à l'instruction 200. (C D I R)
```

En faisant F1 sur le message on obtient les informations sur l’identifiant du message

```
Cause . . . . . : Le programme CL (langage de contrôle) GESTERRCL de la
  bibliothèque MG a trouvé une erreur à l'instruction numéro 200. Le texte du
  message CPF2103 est le suivant : La bibliothèque FORMATION existe déjà dans
  la liste des bibliothèques..
```



Programmation CL

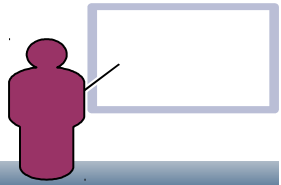
- Interception des messages d'erreur
  - commande de gestion des messages d'erreur MONMSG

```
MONMSG     MSGID(CPFxxxx) N° du ou des messages à monitorer
            EXEC(la commande à exécuter si le message
                est récupéré facultatif)
```

  - ATTENTION cette commande ne monitore que les messages de type ERREUR

```
***************  Début des données  *****
PGM
             ADDLIBLE   LIB(FORMATION)
             MONMSG     MSGID(CPF2103)
ENDPGM
***************  Fin des données  ******
```

  - Si la bibliothèque est déjà dans la liste, le message CPF2103 est intercepté et le traitement continue



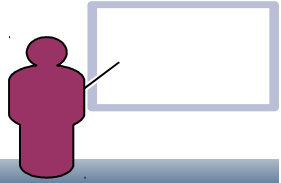
Programmation CL

- Interception des messages d'erreur
  - Il est possible de monitorer un groupe de messages en indiquant une clé générique" (plusieurs messages à la fois).

```
MONMSG CPF2300  monitore les messages 2301 à 2399
MONMSG CPF0000  monitore les messages 0001 à 9999
```

  - ATTENTION si vous définissez vos propres messages ne jamais utiliser de numérotation multiple de 10 (MSG0000 ou MSG1000 etc...)
  - La commande **MONMSG** se place après une commande, il sera pris en compte seulement pour la commande qui le précède (sauf un autre MONMSG), il est possible d'indiquer plusieurs MONMSG pour une même instruction
  - Il est possible de monitorer un ou plusieurs messages sur l'ensemble des instructions d'un programme
    - Il faut placer la commande **MONMSG** avant la première instruction de traitement
    - Le paramètre **EXEC** dans ce cas ne peut contenir que l'instruction GOTO avec un label

```
PGM
  DCL---
  DCL---
  MONMSG MSGID(----)   EXEC(GOTO xxxx)
   ----Traitement
  CALL FINDEMOIS
  ------Traitement
  RETURN /* fin du pgm */
xxxx:
  ENDPGM
```



Programmation CL

Voilà, j'espère que cela vous a intéressé.

Merci.